Speaker Notes: Milestone 2 Video

CSE 516 - g12_th_conditional-ind-graphs_ind - Het Patel, AU2440266

There are 29 slides. At a normal speaking pace of about 150 words a minute the notes take about 17 minutes, plus the live demo. These notes are not inside the slide file.

Slide 1: Title

Hello everyone. I am Het Patel. This is my Milestone 2 video for CSE 516, Probabilistic Graphical Models. My project studies a model called uGLAD. It learns which variables in a dataset are directly linked. In Milestone 1 I explained the problem and the papers. In this video I show the model as code, the maths behind learning and inference, how I checked it, and my first measured results. There is a short live demo in the middle.

Slide 2: What This Talk Covers

This is the plan. First the problem and the model. Then learning and inference, the two things the course asks about, with the main equations explained one term at a time. Then validation, which shows the maths and my code are right. Then the dataset and baseline, and last my first results. I keep results for this milestone small. Bigger experiments are planned for the next milestone.

Slide 3: 1 · The Problem

I have a table of data. Each column is a variable. I want to know which pairs of variables are directly linked. Take three variables X, Z and Y. Suppose X and Y look related only because both depend on Z. Once I know Z, X and Y tell me nothing new about each other. We say they are conditionally independent given Z. The graph that draws an edge only between directly linked variables is called a conditional independence graph.

Slide 4: 1 · Correlation Misleads

Here is a small example with three variables in a chain, X1 to X2 to X3. The left matrix is Theta, the precision matrix. The right matrix is its inverse, the covariance Sigma. In the covariance, X1 and X3 have a correlation of one third, so they look related. But the entry for X1 and X3 in Theta is exactly zero, so they are not directly linked. The partial correlation P is zero for that pair and one half for the real links. I checked these numbers with code.

Slide 5: 2 · The Model

The model is a Gaussian graphical model. The variables together follow a Gaussian distribution with mean zero. Its density depends on the precision matrix Theta, the inverse of the covariance. The key rule is the second equation. If the entry Theta i j is zero, variables i and j are independent once all other variables are known, so there is no edge. All the structure of the graph sits in Theta.

Slide 6: 2 · The Model as Factors

A graphical model writes the joint density as a product of small factors. Here there is one node factor for each variable and one edge factor for each pair. The node factor uses the diagonal entry

of Theta. The edge factor uses the off-diagonal entry. If an entry is zero, its edge factor is exactly one, so the edge disappears. In my code this is the class GaussianGraphicalModel. I checked that the sum of the log factors equals the exact log density up to one constant.

Slide 7: 3 · Learning: The Objective

Learning means estimating Theta from data. The first part of the objective is the negative log likelihood of the data. S is the sample covariance. It measures how well Theta fits. If we only minimized this part, we would need to invert S. That fails when there are fewer samples than variables, and even when it works the answer has no exact zeros. So we add the second part, a penalty on the size of the off-diagonal entries. It pushes small entries to exactly zero. This is the graphical lasso. The strength of the penalty, rho, is usually chosen by cross validation.

Slide 8: 3 · Why the Penalty Gives Zeros

To see why the absolute value gives exact zeros, look at one number. We want the value t closest to a, but we also pay tau times the size of t. The answer is the soft threshold. If the raw value a is smaller than tau in size, the answer is exactly zero. Otherwise the value is moved towards zero by tau. In the plot, tau is 0.3. The four black dots are the raw values minus one, minus 0.2, 0.1 and 0.5. They become minus 0.7, zero, zero and 0.2.

Slide 9: 3 · Solving It: Split in Two

The penalty makes the problem hard to solve directly. The trick used by GLAD and uGLAD is to split Theta into two copies, Theta and Z. Theta takes care of the likelihood. Z takes care of the sparsity. The last term pulls the two copies together with a weight lambda. Then we alternate two easy steps, one for Theta and one for Z. Each step has an exact answer.

Slide 10: 3 · The Θ Step

Fix Z and minimize over Theta. Set the gradient to zero. After multiplying by Theta, you get a quadratic equation in a matrix. The top equation shows that step. Its positive solution is the bottom equation. It uses a matrix square root. I wrote the square root myself in PyTorch and tested its gradient against finite differences. You can check the formula on one number. With lambda two, S 1.5 and Z 0.3, the formula gives 0.517 and the gradient is zero to 16 decimals.

Slide 11: 3 · The Z Step and What Is Learned

The Z step is the soft threshold from before, applied to every entry. In classical graphical lasso the threshold is one fixed number. In uGLAD the threshold rho i j is given by a small neural network. It reads three numbers for that entry: the new Theta value, the sample covariance value and the previous Z value. A second small network gives the next coupling weight lambda. These two networks are shared by all 15 layers. Together they have 42 weights, whatever the number of variables.

Slide 12: 3 · uGLAD: Unroll 15 Steps

The classical method repeats the two steps until it converges, which can take thousands of repeats. uGLAD takes exactly 15 repeats and stacks them like the layers of a network. The input

is the sample covariance S. The output is the matrix after layer 15, which is the graph. Because every step can be differentiated, the whole stack can be trained with gradient descent. The key point is that only 42 weights are learned, so the network can be trained on one dataset.

Slide 13: 3 · One Layer, Step by Step

This is the algorithm. One layer has six steps, in the order of the author's code. Step one forms a helper matrix from the data and the last answer. Step two is the exact Theta step. Step three collects three numbers for every matrix entry. Step four feeds them to the first small network to get a threshold. Step five applies the soft threshold, which creates exact zeros. Step six uses the second small network to choose the next coupling weight. Repeat this 15 times.

Slide 14: 3 · Training Without Labels

The network is trained without any true graph. The loss is the negative log likelihood of the output matrix, computed from the data only. I train with Adam, using the author's defaults: 250 epochs, learning rate 0.002 and 3 folds. In cross validation mode I train on part of the rows. After every epoch I check the same loss on the held-out rows and keep the best epoch. Then I keep the best of the folds and apply it to all the data. The network is trained again from scratch for every new dataset.

Slide 15: 4 · Inference

Inference means answering questions with the learned model. Because the model is Gaussian, every answer is exact and has a closed form. To predict one variable from all the others, the mean is a weighted sum of the others. The weights come straight from the row of Theta. The variance is one over the diagonal entry. The partial correlation gives the strength and sign of a direct link. I checked these formulas against regression on 400000 simulated samples. One query takes about 13 microseconds at 25 variables.

Slide 16: 5 · Validation: Maths Against References

I checked 23 things before running experiments, and all pass. Here are two of them as plots. On the left, the alternating steps never increase their objective, and they reach the same minimum as an independent optimizer. Three coupling weights are shown. On the right, the closed form conditional mean coefficient is compared with the coefficient found by regression on 400000 simulated samples. The points lie on the diagonal. The largest difference over 600 coefficients is 0.005.

Slide 17: 5 · Validation: My Code and the Authors' Code

I also compared my own PyTorch implementation with the authors' public code. Both were run on exactly the same 60 datasets with the same seeds. Each marker is one dataset. The points sit on the diagonal. The correlation of AUPR is 1.00 and the largest difference is 0.003. They are not identical, because I compute the matrix square root with an eigendecomposition in double precision and the authors use an iteration in single precision. But they rank the node pairs in the same way.

Slide 18: 6 · Dataset

I use synthetic Gaussian data, following the protocol of the uGLAD paper and generated with the authors' own generator. First a random graph where every pair gets an edge with probability p. Then random edge values between minus one and one. The generator averages the matrix with its transpose and adds a multiple of the identity so the smallest eigenvalue is one. That is the true precision matrix. Then I draw M samples. I use 25 variables, so about 30 true edges out of 300 pairs. Sample sizes are 10, 25 and 50, with 20 graphs each. Every graph has a fixed seed. There is no real dataset in this milestone.

Slide 19: 6 · Baseline and KPIs

The baseline is the one used in the paper, SGLCV, which is scikit-learn's graphical lasso with the penalty chosen by cross validation. I run uGLAD in two ways, the authors' code and my own implementation. I also show the true model as a reference, because no method can do better than the truth. For the KPIs I measure three layers. For the graph I use AUPR and AUC, which rank all pairs by the size of the learned entry and need no threshold. A random ranking gives an AUPR of about 0.1 here. For learning I use the held-out likelihood. For inference I use the prediction error of the conditional mean.

Slide 20: 7 · Live Demo

Now I run the demo. The command is python demo.py. It makes data from a known model with 25 variables and 50 samples, in the same setting as my experiments. It learns the graph with graphical lasso and with uGLAD, prints the scores, and saves a picture. Then it uses the learned model to answer three queries. This is one run. It shows that the pipeline works from end to end. It is not a benchmark. [Pause the slides here, run the command in a terminal, show the printed output and the saved figure, then return.]

Slide 21: 7 · Result 1: Edge Recovery

This is the main result. It shows AUPR and AUC against the number of samples. Each point is the mean over 20 graphs and the bars show one standard deviation. The hollow dotted markers are the numbers printed in the paper. First look at the blue SGLCV curve. It follows the paper curve closely, so the baseline is reproduced. Now the red curve, which is uGLAD run with the authors' code at its default settings. It does not reach the published values. It is tied with SGLCV at M equal to 10 and behind it at 25 and 50. My own implementation, in green, gives the same result as the authors' code. The red box notes that uGLAD returns an empty graph on 11 of 20 graphs at M equal to 25 and 12 of 20 at M equal to 50. I have not yet tested whether longer or tuned training changes this with the authors' generator. That is the main question for the next milestone.

Slide 22: 7 · Result 2: Graph by Graph

The left plot has one point per graph. A point above the dashed line means uGLAD beat SGLCV on that graph. At M equal to 10 the points are on both sides. At 25 and 50 almost all points are below the line. The right plot shows how many edges each method returned. The true graph has

about 30. Each marker is one graph and the black bar is the mean. At M equal to 10 uGLAD returns far too many edges, about 130 on average. At 25 and 50 about half of its graphs are empty, with zero edges. SGLCV is almost never empty.

Slide 23: 7 · Result 3: One Recovered Graph

Here is one recovered graph so you can see what the numbers mean. I fixed the rule before looking: the largest sample size, M equal to 50, and the first graph. So this example is not picked. The left graph is the truth with its edges. In the middle SGLCV finds a few of the edges. On the right uGLAD returns an empty graph for this dataset, so all the true edges are missed. Solid black is a correct edge, dashed red is an extra edge and dotted gray is a missed edge.

Slide 24: 7 · Result 4: Likelihood and Prediction

These are the learning and inference KPIs. They use 1000 fresh test samples from the true model, and all methods are compared in the raw units of the data. The green curve is the true model. No method can go below it. On the left, the held-out negative log likelihood. SGLCV gets close to the truth as M grows. uGLAD is worse, which matches the empty graphs. On the right, the error when predicting each variable from all others. All methods are close to the truth here. This means the prediction error alone cannot tell a good graph from a poor one, so I report the edge scores as well.

Slide 25: 7 · Table 1 Against the Paper

This table puts the numbers behind the plots next to the paper. The left table is AUPR and the right is AUC. For each method the first row is the paper and the next rows are my runs. SGLCV agrees with the paper within one standard deviation. uGLAD with the authors' code, and my implementation, are below the published uGLAD numbers and agree with each other. I use the same graph sizes and 20 graphs like the paper. So the baseline is reproduced and the uGLAD claim at default settings is not.

Slide 26: 8 · What I Learned

Three findings. First, the baseline is reproduced. SGLCV agrees with the paper within one standard deviation. Second, uGLAD at the code defaults does not show the published advantage. It ties with SGLCV at M equal to 10 and is behind at 25 and 50, often returning an empty graph. Third, my own implementation agrees with the authors' code on 60 datasets. So the finding comes from the method at these settings and not from a bug in my code. I do not claim yet that uGLAD cannot work. Training details are not fully given in the paper.

Slide 27: 8 · Limits

These are the limits of this milestone. I only ran the paper's Table 1 setting. I used 20 graphs and the default settings of the code. The paper gives a learning rate range that differs from the code default and does not give the number of epochs, so I cannot say the paper's exact training was matched. All data is synthetic and Gaussian. So these results are preliminary and I treat them as a first measurement.

Slide 28: 8 · Planned for M3

This is my plan and it may change. The open question from this milestone is why uGLAD at default settings is behind. So in the next milestone I want to study the training length and learning rate. I also want larger sample sizes and the other baselines from the course, PC, GES and hill climbing. Then a rule to turn the learned matrix into a graph, noise and missing data, and significance tests over more graphs. I deliberately keep these out of this milestone so that each milestone has its own results.

Slide 29: References

These are the references used in the slides. The full list is in the report. I used only sources that I checked.